Отчёт по модулю «Защита БД и целостность (SQLite)» (module03)

Репозиторий: https://github.com/Teni-Moridera/module03. Содержание отчёта согласовано с `README.md`, `ОТЧЕТ_SQLITE.md`, `ПОЛИТИКА_БЕЗОПАСНОСТИ_SQLITE.md` и SQL-скриптами в каталоге `sqlite/`.

1. Цель модуля

Отработать на встраиваемой **SQLite** комплекс мер: **целостность данных**, **журнал аудита** и **триггеры**, **логическую модель ролей и пользователей** с явной **политикой доступа** (в рамках ограничений SQLite), а также сценарии **резервного копирования** и воспроизводимой сборки БД из скриптов.

2. Постановка задач (что требовалось сделать)

№	Задача	Смысл
1	Спроектировать предметную область (≥4 таблицы + служебные)	Модель «книжный магазин» и учёт ролей/пользователей
2	Включить целостность на уровне схемы	FK, CHECK, UNIQUE, индексы
3	Описать разграничение доступа без GRANT	Таблицы `roles`/`db_users`, представления, документ политики
4	Реализовать аудит изменений	Таблица `audit_log` и триггеры на изменяемые сущности
5	Добавить триггеры предметной логики	Контроль остатка и запрет некорректных значений
6	Заполнить роли и демо-данные	Воспроизводимая демонстрация
7	Автоматизировать создание и бэкап БД	Скрипты PowerShell для CLI `sqlite3`
8	Зафиксировать ограничения модели SQLite	Честное описание: физическая защита файла, права в приложении

3. Что сделано и зачем

3.1. Схема (`sqlite/01_schema.sql`)

- Предметная область: **authors**, **books**, **customers**, **orders**, **order_items** с внешними ключами и проверками (цена, остаток, статус заказа и т.д.).
- Служебные таблицы: **roles**, **db_users** (логин, заглушка хэша пароля, связь с ролью), **audit_log** для журнала.
- Прагмы: `foreign_keys = ON`, `journal_mode = WAL`, `synchronous = NORMAL` — баланс целостности и производительности при журналировании.

Зачем: единый файл БД `sqlite/bookstore_lab.db` с явной моделью данных и подготовкой к триггерам и аудиту.

3.2. Триггеры (`sqlite/02_triggers.sql`)

- Триггер `trg_books_stock_check`: запрет отрицательного `stock_qty` на книгах.
- После INSERT/UPDATE/DELETE на `books`, `orders`, `order_items` — записи в `audit_log` с операцией, первичным ключом и снимками строк в виде `json_object` (OLD/NEW).

Зачем: бизнес-инвариант по складу и неизменяемый след изменений в таблицах заказов и каталога.

3.3. Роли и пользователи (`sqlite/03_seed_roles.sql`)

- Роли: `admin`, `librarian`, `sales`, `readonly` с описаниями; примеры строк в `db_users` с заглушками `password_hash`.

Зачем: логическая модель «кто есть кто» для приложения или учебных сценариев; в реальной системе проверка пароля — на стороне клиента, не в движке SQLite.

3.4. Представления доступа (`sqlite/04_views_access.sql`)

- `v_readonly_catalog` — книги с автором; `v_readonly_orders` — заказы с данными клиента; `v_audit_readonly` — чтение журнала аудита.

Зачем: сценарий «только чтение» через согласованные объекты, т.к. в SQLite нет `GRANT` на уровне сервера.

3.5. Демо-данные и индексы (`sqlite/05_demo_data.sql`, `sqlite/06_integrity_extra.sql`)

- Наполнение для проверки запросов и связей.
- Дополнительные индексы по внешним ключам и по времени в `audit_log`.

Зачем: ускорение типичных соединений и выборок из журнала.

3.6. Политика безопасности (`ПОЛИТИКА_БЕЗОПАСНОСТИ_SQLITE.md`)

- Зафиксированы ограничения встраиваемой СУБД: **физическая защита файла БД**, матрица целевого поведения ролей (каталог vs заказы vs аудит), пояснение, что жёсткое принуждение ролями только средствами SQLite без приложения невозможно.

Зачем: документировать честную модель угроз и ожидаемое использование объектов БД.

3.7. Скрипты (`scripts/init_sqlite.ps1`, `scripts/backup_sqlite.ps1`)

- **init**: удаление старого файла БД, последовательное выполнение `01…06` через `sqlite3 … ".read …"`.
- **backup**: согласованная копия через команду `.backup` в каталог `backups/` с меткой времени.

Зачем: воспроизводимая сборка и безопасное резервирование при WAL без ручного копирования «на лету» без учёта блокировок.

4. Технологии и обоснование выбора

Технология	Зачем использована
SQLite	Встраиваемая СУБД в одном файле: удобно для учебного проекта, лабораторных и демонстрации целостности и триггеров без отдельного сервера.
SQL (DDL/DML)	Явное описание схемы, версионируемое в репозитории.
Триггеры SQLite	Аудит и проверки на уровне БД; для `books`/`orders`/`order_items` — отдельные триггеры под INSERT/UPDATE/DELETE из-за правил OLD/NEW.
JSON через `json_object`	Компактное хранение снимков строк в `audit_log` без дополнительных таблиц под каждую сущность.
Представления (VIEW)	Имитация слоя «только чтение» там, где нет серверных ролей SQL.
PowerShell + sqlite3 CLI	Автоматизация на Windows: быстрый старт из README и согласованный бэкап через встроенную команду `.backup`.
WAL-режим	Улучшает параллельность чтения при записи; для бэкапа важно использовать API копирования, а не слепое копирование файла при активной записи.

5. Краткий вывод

В модуле реализована учебная БД «книжный магазин» с **FK, CHECK, индексами**, **триггерным аудитом** и контролем остатка, **логическими ролями** и **представлениями** под ограничения SQLite, плюс **документированная политика доступа** и **скрипты инициализации и резервного копирования**. Выбор SQLite обоснован задачей показать целостность и журналирование в компактной среде; серверные механизмы вроде `GRANT` здесь заменены сочетанием **физической защиты файла**, **соглашений в приложении** и **представлений**, что прямо отражено в репозитории.